AEGISAI ENTERPRISE EDITION

SUITE DE DOCUMENTACIÓN TÉCNICA

Documento de Arquitectura de Software

Arquitectura Desacoplada, Modelado de Amenazas y Cumplimiento OWASP AI-SVS + ISO/IEC 42001

Versión de Plataforma:	**2.5.0 Enterprise Edition**
Alcance Normativo:	**OWASP AISVS v1.0, ISO/IEC 42001:2023, Google Cloud Security**
Licencia:	**Apache License 2.0 (Open Source)**

Equipo de Ingeniería AegisAI

Clasificación: Público / Open Source

AEGIS AI ASSURANCE PLATFORM

Documento de Arquitectura de Software (Software Architecture Document - SAD)

Versión: 2.5.0 Enterprise (Edición 2026)

Estándar de Documentación: Basado en IEEE 1471 / ISO/IEC 42010 y Modelo 4+1 Vistas

Clasificación: Documento Técnico de Arquitectura y Diseño para Ingenieros Principales y Auditores

---

1. Introducción y Metas de Arquitectura

1.1 Propósito

Este documento describe formalmente la arquitectura de software de **AegisAI**, una plataforma de misión crítica orientada al aseguramiento de calidad, ciberseguridad y cumplimiento normativo en sistemas de Inteligencia Artificial Generativa y Machine Learning.

1.2 Atributos de Calidad Principales (Quality Attributes)

1. **Hiper-Factualidad Determinista (QA-01):** Cero tolerancia a alucinaciones. La IA debe evaluar exclusivamente con base en la evidencia documental provista, rechazando inferencias no fundamentadas.
2. **Capacidad de Salida Completa sin Truncamiento (QA-02):** Capacidad de auditar más de 40 controles técnicos exhaustivos sin sobrepasar el límite estricto de 8,192 tokens de salida por llamada de los LLMs.
3. **Inmutabilidad y Trazabilidad Forense (QA-03):** Cumplimiento del Invariante 4 de OWASP AI-SVS: los registros de auditoría no pueden ser alterados ni borrados una vez dictaminados.
4. **Resiliencia y Alta Disponibilidad (QA-04):** Tolerancia a fallos ante caídas transitorias de red, límites de cuota de proveedores de nube o saturación de modelos.
5. **Rendimiento y Baja Latencia (QA-05):** Ingestión de normativas legales y procesamiento en tiempo real con latencias sub-segundo en tareas estructuradas.

---

2. Patrones de Diseño y Decisiones Arquitectónicas

2.1 Patrón Map-Reduce para Inferencia Distribuida

- **Problema:** Los modelos LLM contemporáneos disponen de ventanas de entrada masivas (1M a 2M tokens), pero mantienen un límite rígido de generación de salida (**completion tokens**) de 8,192 tokens. Al intentar auditar 12 capítulos (44 controles) con recetas de mitigación completas en una sola petición, el modelo comprime o trunca drásticamente las respuestas.
- **Solución:** Se implementó una arquitectura Map-Reduce en `server.ts`:
- **Fase MAP:** Los capítulos del estándar se particionan en 3 o 4 lotes paralelos (ej. Lote 1: C01-C03; Lote 2: C04-C06; Lote 3: C07-C09; Lote 4: C10-C12). Cada lote se envía de forma independiente al LLM con el esquema estructurado estricto.
- **Fase REDUCE:** El orquestador unifica en memoria las matrices de resultados, calcula la calificación ponderada general y consolida las no conformidades en el objeto final `AuditReport`.

2.2 Patrón de Streaming Server-Sent Events (SSE)

- **Problema:** Una evaluación técnica exhaustiva de más de 40 controles toma entre 20 y 45 segundos. Una llamada HTTP síncrona tradicional provocaría bloqueos de interfaz o timeouts en proxies.
- **Solución:** La ruta `/api/evaluate` opera bajo el protocolo SSE (`text/event-stream`), despachando eventos de progreso granulares (`{ type: 'progress', progress: 50, message: '...' }`) y manteniendo la conexión abierta con pulsos `: keep-alive\n\n` cada 12 segundos.

2.3 Patrón de Persistencia Dual Resiliente

- **Problema:** Depender exclusivamente de una base de datos en la nube (Cloud Firestore) expone la aplicación a interrupciones por propagación de reglas de seguridad, desconexiones o bloqueos de permisos.
- **Solución:** Todo marco normativo o auditoría se almacena de inmediato en el sistema de archivos local (`config/frameworks/{id}.json`) y se replica de forma asíncrona y no bloqueante en Firestore. Si Firestore devuelve `PERMISSION_DENIED` o falla de red, el sistema continúa operando al 100% desde la caché local.

2.4 Patrón de Deduplicación por Responsabilidad Compartida (Shared Responsibility)

- **Problema:** En arquitecturas desplegadas sobre Google Gemini Enterprise SaaS, penalizar al cliente por no demostrar cómo cifra los pesos del modelo en hardware TPU físico es un error de auditoría que desvía la atención de la seguridad de la aplicación.

2.5 Patrón de Desacoplamiento Multi-Proveedor e Inferencia Universal

- **Problema:** Forzar la dependencia exclusiva de un único proveedor de IA (vendor lock-in) impide a clientes corporativos utilizar sus acuerdos existentes con OpenRouter, OpenAI, o servidores locales de inferencia (vLLM, Ollama, LiteLLM).
- **Solución:** Se implementó un motor desacoplado centrado en `executeAIWithFallback` :
- Si el proveedor configurado es `gemini` , utiliza el SDK `@google/genai` con su jerarquía de resiliencia (`gemini-3.8-flash` -> `gemini-3.1-flash-lite` -> `gemini-3.7-flash`).
- Si el proveedor es `openrouter` , `openai` o `custom_openai_compatible` , utiliza un cliente HTTP nativo que implementa la especificación universal `/v1/chat/completions` con timeouts de 120s, backoff exponencial, extracción estricta de JSON y accounting de tokens de entrada y salida.

2.6 Patrón de Generación de Arquitectura Remediada (Drop-in Replacement)

- **Problema:** Entregar a un cliente únicamente un reporte con calificaciones reprobatorias o listas de no conformidades no resuelve la brecha técnica y genera fricción operativa.
- **Solución:** Cuando se detectan fallas o falta de evidencia, el sistema orquesta una fase de remediación sintética. El motor toma la arquitectura original del usuario y teje todas las salvaguardas faltantes (sanitización anti-inyección, anonimización DLP, aislamiento RAG, logs inmutables y escalamiento humano), produciendo un documento Markdown (`.md` / `.txt`) listo para producción que garantiza el **100.0% de cumplimiento normativo**.

2.7 Patrón de Orquestación Multi-Normativa Simultánea (Compliance 360°)

- **Problema:** En entornos de producción y sectores regulados (banca, retail, salud), un sistema de IA debe satisfacer simultáneamente múltiples capas normativas: ciberseguridad técnica (OWASP AI-SVS 1.0), gobernanza de IA y gestión de riesgos (ISO/IEC 42001:2023), privacidad de datos (LFPDPPP / INAI) y transparencia comercial (LFPC / PROFECO). Ejecutar auditorías fragmentadas multiplica la latencia y genera silos de reporte.
- **Solución:** `runArchitectureEvaluation` soporta la especificación concurrente de múltiples marcos normativos mediante `selectedStandards: string[]` . Durante la fase MAP, el motor agrega los lotes de todos los estándares seleccionados, inyectando metadatos de linaje (`standardId` , `standardName` , `contextText`) en cada bloque de inferencia. En la fase REDUCE, el sistema calcula la métrica global y estructura el arreglo `standardsBreakdown` ,

donde cada estándar preserva su calificación independiente (0-100%), total de controles evaluados y estado de cumplimiento, permitiendo renderizar tableros de Compliance 360° y exportar reportes ejecutivos consolidados.

2.8 Patrón de Inspección en Vivo de Infraestructura Cloud (GCP Live Scanner - Zero-Footprint)

- **Problema:** Auditar exclusivamente documentos estáticos o diagramas escritos introduce sesgos humanos ("wishful thinking" o discrepancias entre la especificación y la implementación real). Asimismo, desplegar agentes de software o funciones en el proyecto del cliente genera fricción con áreas de ciberseguridad corporativa por costos, riesgos de estabilidad y procesos de aprobación.
- **Solución:** Se diseñó el servicio `gcpScannerService`, que interactúa exclusivamente con el **Plano de Control de Google Cloud (Management APIs)** mediante consultas `HTTP GET` de solo lectura. El servicio soporta autenticación en memoria mediante ADC (Cloud Run), tokens temporales Bearer o Cuentas de Servicio sin persistencia de llaves en disco. Inspecciona configuraciones críticas de Cloud IAM, Vertex AI, Cloud Run, Cloud Storage y Secret Manager, sintetizando un manifiesto técnico estructurado ( ) que se alimenta directamente al motor Map-Reduce multi-normativo, garantizando una auditoría basada en la realidad viva de la nube con **cero huella y cero recursos creados en el cliente**.

---

3. Vista de Componentes Lógicos

```
flowchart TD
    subgraph ClientLayer ["Capa de Cliente (Frontend SPA)"]
        UI["React 19 + Tailwind CSS v4\nCode-Splitting por Vistas"]
        SSE_Client["SSE Event Consumer\nStreaming Reactivo"]
        PDF_Engine["jsPDF + AutoTable\nCarga Dinámica Bajo Demanda"]
        RemediationUI["Modal de Vista Previa y\nDescarga de Arquitectura Remediada"]
    end

    subgraph GatewayLayer ["Capa de Enrutamiento y Seguridad (Express)"]
        SecHeaders["Cabeceras OWASP\n(nosniff, SAMEORIGIN, PermPolicy)"]
        DotfileBlocker["Filtro Anti-Sondeo\n(Bloqueo de /.env, .key, .git)"]
        AuthMiddleware["Autenticación RBAC\n(x-admin-token, x-api-key)"]
    end

    subgraph CoreEngine ["Núcleo Evaluador de IA Multi-Proveedor"]
        Orchestrator["runArchitectureEvaluation\n(Orquestador Map-Reduce)"]
        Decomposer["decomposeLegalDocument\n(Motor Universal BYOF)"]
        Remediator["remediatedArchitectureGenerator\n(Síntesis de 100% Cumplimiento)"]
        Dispatcher["executeAIWithFallback\n(Despachador Central)"]
        GeminiClient["Google GenAI SDK\n(Fallback Chain 503/429)"]
        UniversalOpenAI["Cliente Universal /v1/chat/completions\n(OpenRouter / OpenAI / vLLM / Ollam
    end

    subgraph PersistenceLayer ["Capa de Persistencia Dual"]
        LocalFS["Sistema de Archivos Local\nconfig/frameworks/*.json & admin-settings.json"]
        CloudFirestore["Google Cloud Firestore\nColecciones: audits, custom_frameworks"]
    end

    UI --> SecHeaders
    SecHeaders --> DotfileBlocker
    DotfileBlocker --> AuthMiddleware
    AuthMiddleware --> Orchestrator
    AuthMiddleware --> Decomposer
    AuthMiddleware --> Remediator
    Orchestrator --> Dispatcher
    Decomposer --> Dispatcher
    Remediator --> Dispatcher
    Dispatcher -->|gemini| GeminiClient
    Dispatcher -->|openrouter/openai/local| UniversalOpenAI
    Orchestrator --> LocalFS
    Orchestrator --> CloudFirestore
    Decomposer --> LocalFS
    Decomposer --> CloudFirestore
    Orchestrator -. Eventos SSE .-> SSE_Client
    Remediator -. Especificación .md .-> RemediationUI
```

4. Modelos de Dominio y Datos

4.1 Entidad `AuditReport`

Estructura representativa del dictamen técnico emitido:

```
interface AuditReport {
  id: string;                     // Identificador unívoco (ej. LFPDPPP-2026-0001)
  date: string;                   // Marca temporal ISO 8601
  systemName: string;             // Nombre del sistema auditado
  technicalLead: string;          // Líder técnico responsable
  email: string;                  // Correo de contacto
  description: string;            // Especificación técnica evaluada
  standard: 'AI-SVS' | 'ISO-42001' | 'FULL' | 'CUSTOM';
  customFrameworkId?: string;     // ID de la ley personalizada si standard === 'CUSTOM'
  customFrameworkName?: string;   // Nombre descriptivo de la regulación
  targetLevel: 'L1' | 'L2' | 'L3' | 'AUTO';
  assessedLevel: 'L1' | 'L2' | 'L3';
  levelAssessmentRationale: string;// Justificación técnica del nivel
  techPlatform: string;           // gemini-enterprise, vertex-ai, self-hosted
  techPlatformName: string;       // Nombre comercial de la plataforma
  deploymentType: 'SaaS' | 'PaaS' | 'IaaS' | 'Self-Hosted';
  overallScore: number;           // Calificación ponderada (0-100%)
  executiveSummary: string;       // Resumen ejecutivo de riesgos
  chapters: StandardChapter[];    // Capítulos evaluados con sus puntos probados
  modelUsed: string;              // Modelo exacto de Gemini utilizado
  latencyMs: number;              // Tiempo total de inferencia
  promptTokens: number;           // Tokens de entrada
  completionTokens: number;       // Tokens de salida generados
}
```

4.2 Entidad `PuntoProbado` y Receta 4D de Mitigación

```
interface PuntoProbado {
  id: string;                     // Identificador del control (ej. LFPDPPP-03.1)
  name: string;                   // Título del requisito
  status: 'Aprobado' | 'Requiere Revisión' | 'Falta Evidencia' | 'No Aplica';
  findings: string;               // Hallazgos y justificación técnica
  fuenteEvidencia?: string;       // Cita textual exacta de la documentación
  isInheritedFromPlatform?: boolean;// True si está cubierto por Google Cloud SaaS
  inheritedPlatformName?: string;  // Nombre de la nube acreditada
  level?: 'L1' | 'L2' | 'L3';     // Nivel de verificación
  isMandatoryForTargetLevel?: boolean;
  remediationDetails?: {
    strategy: string;             // Dimensión 1: Estrategia arquitectónica
    actionableSteps: string[];    // Dimensión 2: Checklist paso a paso
    codeOrConfigExample: string;  // Dimensión 3: Código o manifiesto real
    verificationRecipe: string;   // Dimensión 4: Script de pentesting / curl QA
    effortLevel: string;          // Esfuerzo estimado (< 1 día, 1-3 días, etc.)
    recommendedTools: string[];   // Librerías recomendadas (Presidio, Sigstore)
  };
}
```

5. Vista de Procesos: Diagramas de Secuencia

5.1 Flujo de Auditoría con Map-Reduce y Streaming SSE

```
sequenceDiagram
    autonumber
    actor User as Auditor / Usuario
    participant FE as Frontend React
    participant API as Backend Express (/api/evaluate)
    participant Engine as runArchitectureEvaluation
    participant Gemini as Google Gemini AI Studio
    participant Storage as Persistencia Dual

    User->>FE: Envía Formulario de Auditoría (Especificación + Plataforma)
    FE->>API: POST /api/evaluate (SSE Handshake)
    API-->>FE: HTTP 200 (text/event-stream conectado)
    API->>Engine: Inicia Evaluación

    loop Para cada Lote Map (Lotes 1 a 4)
        Engine->>Gemini: generateContent(Batch Prompt + Schema + Responsabilidad Compartida)
        Gemini-->>Engine: JSON Estructurado con Controles y Citas Textuales
        Engine-->>FE: SSE Event (progress: XX%, lote completado)
    end

    Engine->>Engine: REDUCE (Consolidación de puntuaciones y hallazgos)
    Engine->>Storage: Guarda en local config/frameworks/ y Cloud Firestore
    Engine-->>FE: SSE Event (type: 'result', data: AuditReport)
    FE->>User: Renderiza Reporte con Badges de Herencia y Recetas 4D
```

---

6. Modelo de Amenazas y Seguridad (Threat Modeling)

AegisAI ha sido sometido y blindado frente a las 10 categorías de amenazas de **OWASP Top 10 for LLM Applications**:

Identificador	Vector de Amenaza	Mecanismo de Defensa Implementado en AegisAI
LLM01	Prompt Injection	Delimitadores estrictos `<user_candidate_architecture>`, parámetros deterministas (`temp: 0.0`) y aislamiento de datos de entrada como no confiables.
LLM02	Insecure Output Handling	Tipado rígido en esquemas JSON, sanitización contra XSS en React y renderizado seguro en exportadores de PDF y Markdown.
LLM03	Data Poisoning	Validación estructural obligatoria en la ingesta de normativas (`frameworkDecomposerSchema`).
LLM04	Model Denial of Service & Denial of Wallet (`C11.1`)	Límite máximo de payload en Express (30MB) con rechazo HTTP 413, más limitador de tasa por identidad (`10 req/min`), tope de concurrencia (`2 auditorías activas/usuario`) y presupuesto rodante de tokens (`500,000 tokens/hora`) con respuesta `HTTP 429` y cabecera `Retry-After`.
LLM05	Supply Chain & MCP Integrity (`C01.4`, `C10.1`)	Dependencias reducidas, bloqueo en `package-lock.json`, manifiesto criptográfico SHA-256 (`integrity-manifest.json`) y firma HMAC-SHA256 para conectores MCP (`/api/mcp/verify`).
LLM06	Information Disclosure & Model Extraction (`C11.4`)	Máscaras automáticas en logs (`redactSensitiveStrings`) y supresión explícita de probabilidades/logits (`responseLogprobs: false`).
LLM07	Insecure Plugin Design (`C05.3`, `C10.5`)	Comparación en tiempo constante (`crypto.timingSafeEqual`), propagación de identidad Firebase/IAP, guardia contra DNS Rebinding y bloqueo SSRF (`validateOutboundEndpoint`).
LLM08	Excessive Agency (`C05.2`, `C09.4`)	Invariante 4 de OWASP AI-SVS: Auditorías inmutables (`allow delete: if false;`), identidad `aegis-runner-sa` con mínimo privilegio estricto (`aiplatform.user` y `datastore.user`) y saga de compensación transaccional (`runCompensatingSaga`).
LLM09	Overreliance (Alucinación — `C08.4`)	Obligatoriedad de citas textuales literales y telemetría continua de fundamentación citatriz (`/api/admin/aims-telemetry`, SLO $\ge 95\%$).
LLM10	Model Theft / File Probing (`C04.1`)	Sistema de archivos de contenedor de solo lectura (`chmod -R a-w /app`), usuario sin privilegios `aegisuser` y bloqueo HTTP 404 de dotfiles (`/.env`, `.key`, `.git`).

7. Vista de Despliegue en la Nube (Cloud Deployment View)

La arquitectura de infraestructura de AegisAI aprovecha el ecosistema administrado de **Google Cloud Platform (GCP)** para garantizar alta disponibilidad (99.95%), cero mantenimiento de servidores y aislamiento estricto de red Zero-Trust:

```
flowchart LR
    subgraph ClientPerimeter ["Perímetro del Cliente / Auditor"]
        Browser["Navegador Seguro\n(Chrome / Safari / Firefox)"]
    end

    subgraph GCP_Project ["Proyecto Google Cloud (Tenant Enterprise: computeengine-506321)"]
        subgraph IngressLayer ["Capa de Entrada Zero-Trust (C04.3)"]
            CloudIAP["Cloud Identity-Aware Proxy (IAP)\n(--no-allow-unauthenticated / OAuth 2.0)"]
            CloudRunFront["Google Cloud Run Ingress\n(HTTPS TLS 1.3 / Port 8080)"]
            FirebaseAuth["Firebase Auth / Identity Platform\n(Filtro OAuth: google.com, altostrat.co
        end

        subgraph ComputeLayer ["Capa de Cómputo Serverless Inmutable (C04.1, C05.2)"]
            Container["Contenedor AegisAI (Solo Lectura /app: chmod a-w)\n(Node.js 22 LTS, Runner s
            ServiceAccount["Runtime SA: aegis-runner-sa\nMínimo Privilegio Estricto:\naiplatform.use
        end

        subgraph DataAndAILayer ["Servicios de Persistencia, Secretos e IA Gestionados"]
            FirestoreDB["Cloud Firestore Nativo (aegis-ai-db)\nReglas Criptográficas firestore.rules
            VertexAI["Google Cloud Vertex AI (us-central1)\nModelos Gemini 3.8 Flash / 3.1 Flash-Li
            ArtifactReg["Artifact Registry (Docker)\ncloud-run-source-deploy"]
        end
    end

    Browser -->|HTTPS TLS 1.3| CloudIAP
    CloudIAP -->|Identidad Verificada| CloudRunFront
    Browser -.->|OAuth Popup + ID Token| FirebaseAuth
    CloudRunFront --> Container
    Container --- ServiceAccount
    ServiceAccount -->|Lectura/Escritura IAM| FirestoreDB
    ServiceAccount -->|Inferencia ADC| VertexAI
    Container -.->|Despliegue de Imagen| ArtifactReg
```

7.1 Componentes de Infraestructura y Mapeo de Servicios

Componente GCP	Servicio Técnico	Rol y Responsabilidad
Perímetro Zero-Trust	Cloud Identity-Aware Proxy (IAP) & Cloud Run	Aplica `--no-allow-unauthenticated` en el borde (`C04.3`), permitiendo solo cuentas corporativas autorizadas (`google.com`, `altostrat.com`).
Cómputo Serverless	Google Cloud Run (`aegis-ai`)	Aloja el contenedor inmutable (`chmod -R a-w /app`, `C04.1`) con `--vpc-egress private-ranges-only` y protección SSRF (`C04.3`).
Identidad de Ejecución	Cloud IAM (`aegis-runner-sa`)	Cuenta de servicio con mínimo privilegio estricto: exclusivamente `roles/aiplatform.user` y `roles/datastore.user` (`C05.2`).
Base de Datos NoSQL	Cloud Firestore Nativo (`aegis-ai-db`)	Almacena reportes inmutables y catálogos protegidos por `firestore.rules` con vinculación de propietario y tenant (`C05.4`).
Inferencia e IA	Vertex AI (`us-central1`)	Ejecuta consultas Map-Reduce deterministas (`temperature: 0.0`, `responseLogprobs: false`, `C11.4`) vía ADC.
Automatización CI/CD	Google Cloud Build	Orquesta la compilación en la nube con cuenta dedicada `aegis-build-sa`.